Online Banking Analysis using Apache Spark

This project aims to analyze large-scale online banking data using **Apache Spark (PySpark)** for processing, the relational data source, and **Jupyter Notebook** for development and visualization.

🛠️ Tools and Technologies Used:

- **PySpark** – for distributed data processing and analytics
- **Jupyter Notebook** – for writing, running, and visualizing code

📊 Datasets Involved:

1. **Loan Data (`loan.csv`)** – Information about borrowers, loan categories, income, bounced cheques, and spending patterns.
2. **Credit Card Data (`credit card.csv`)** – Contains credit eligibility, country of residence, and user activity status.
3. **Transactions Data (`txn.csv`)** – Includes deposit and withdrawal details, balance per transaction, and transaction dates.

Step 1: Initialize Spark Session and Load Datasets

```
from pyspark.sql import SparkSession

# Start Spark Session
spark = SparkSession.builder.appName("OnlineBankingAnalysis").getOrCreate()

# Load all datasets
loan_df = spark.read.csv("/content/loan.csv", header=True, inferSchema=True)
credit_df = spark.read.csv("/content/credit card.csv", header=True, inferSchema=True)
txn_df = spark.read.csv("/content/txn.csv", header=True, inferSchema=True)
```

Step 2: Clean Column Names

```
# Clean loan_df
loan_df = loan_df.withColumnRenamed("Loan Category", "loan_category") \
                 .withColumnRenamed("Loan Amount", "loan_amount") \
                 .withColumnRenamed("Income", "income") \
                 .withColumnRenamed("Returned Cheques", "returned_cheques") \
                 .withColumnRenamed("Marital Status", "marital_status") \
                 .withColumnRenamed("Monthly Expenditure", "monthly_expenditure") \
                 .withColumnRenamed("Eligible For Credit Card", "eligible_credit_card")

# Clean credit_df
credit_df = credit_df.withColumnRenamed("Country", "country") \
                     .withColumnRenamed("Active", "active") \
                     .withColumnRenamed("Eligible", "eligible")

# Clean txn_df
txn_df = txn_df.withColumnRenamed("Account No", "account_no") \
               .withColumnRenamed("Transaction Type", "transaction_type") \
               .withColumnRenamed("Transaction Amount", "transaction_amount") \
               .withColumnRenamed("Balance", "balance") \
               .withColumnRenamed("Transaction Date", "transaction_date")
```

Step 3: Handle Null Values

```
from pyspark.sql.functions import col

print("🔹 Loan Data Nulls:")
loan_df.select([col(c).isNull().alias(c) for c in loan_df.columns]).show(5)

print("🔹 Credit Card Data Nulls:")
credit_df.select([col(c).isNull().alias(c) for c in credit_df.columns]).show(5)

print("🔹 Transaction Data Nulls:")
txn_df.select([col(c).isNull().alias(c) for c in txn_df.columns]).show(5)
```

```
🔹 Loan Data Nulls:
+-----------+-----+------+----------+--------------+-----------+------+-----------+------------+-------------+-----------+-------+-----------+----------------+-----------------+
|Customer_ID|  Age|Gender|Occupation|marital_status|Family Size|income|Expenditure|Use Frequency|loan_category|loan_amount|Overdue| Debt Record| Returned Cheque| Dishonour of Bill|
+-----------+-----+------+----------+--------------+-----------+------+-----------+------------+-------------+-----------+-------+-----------+----------------+-----------------+
|      false|false| false|     false|         false|      false| false|      false|       false|        false|      false|  false|      false|           false|            false|
|      false|false| false|     false|         false|      false| false|      false|       false|        false|      false|  false|      false|           false|            false|
|      false|false| false|     false|         false|      false| false|      false|       false|        false|      false|  false|      false|           false|            false|
|      false|false| false|     false|         false|      false| false|      false|       false|        false|      false|  false|      false|           false|            false|
|      false|false| false|     false|         false|      false| false|      false|       false|        false|      false|  false|      false|           false|            false|
+-----------+-----+------+----------+--------------+-----------+------+-----------+------------+-------------+-----------+-------+-----------+----------------+-----------------+
only showing top 5 rows

🔹 Credit Card Data Nulls:
+---------+----------+-------+-----------+---------+------+-----+------+-------+-------------+--------------+---------------+------+
|RowNumber|CustomerId|Surname|CreditScore|Geography|Gender|  Age|Tenure|Balance|NumOfProducts|IsActiveMember|EstimatedSalary|Exited|
+---------+----------+-------+-----------+---------+------+-----+------+-------+-------------+--------------+---------------+------+
|    false|     false|  false|      false|    false| false|false| false|  false|        false|         false|          false| false|
|    false|     false|  false|      false|    false| false|false| false|  false|        false|         false|          false| false|
|    false|     false|  false|      false|    false| false|false| false|  false|        false|         false|          false| false|
|    false|     false|  false|      false|    false| false|false| false|  false|        false|         false|          false| false|
|    false|     false|  false|      false|    false| false|false| false|  false|        false|         false|          false| false|
+---------+----------+-------+-----------+---------+------+-----+------+-------+-------------+--------------+---------------+------+
only showing top 5 rows

🔹 Transaction Data Nulls:
+----------+-------------------+----------+---------------+-------------+-----------+
|account_no|TRANSACTION DETAILS|VALUE DATE| WITHDRAWAL AMT | DEPOSIT AMT |BALANCE AMT|
+----------+-------------------+----------+---------------+-------------+-----------+
|     false|              false|     false|           true|        false|      false|
|     false|              false|     false|           true|        false|      false|
|     false|              false|     false|           true|        false|      false|
|     false|              false|     false|           true|        false|      false|
|     false|              false|     false|           true|        false|      false|
+----------+-------------------+----------+---------------+-------------+-----------+
only showing top 5 rows
```

IN LOAN DATA:

1. Number of loans in each category

```
print(" 🔹 Number of loans in each category:")
loan_df.groupBy("loan_category").count().show()
```

```
 🔹 Number of loans in each category:
+------------------+-----+
|     loan_category|count|
+------------------+-----+
|           HOUSING|   67|
|        TRAVELLING|   53|
|       BOOK STORES|    7|
|       AGRICULTURE|   12|
|         GOLD LOAN|   77|
|  EDUCATIONAL LOAN|   20|
|        AUTOMOBILE|   60|
|          BUSINESS|   24|
|COMPUTER SOFTWARES|   35|
|           DINNING|   14|
|          SHOPPING|   35|
|       RESTAURANTS|   41|
|       ELECTRONICS|   14|
|          BUILDING|    7|
|        RESTAURANT|   20|
|   HOME APPLIANCES|   14|
+------------------+-----+
```

2. Number of people who have taken more than 1 lakh loan

```
print(" 🔹 People with loan amount > 1,00,000:")
loan_df.filter(col("loan_amount") > 100000).count()
```

```
 🔹 People with loan amount > 1,00,000:
0
```

3. Number of people with income greater than ₹60,000

```
print(" 🔹 People with income > 60,000:")
loan_df.filter(col("income") > 60000).count()
```

```
 🔹 People with income > 60,000:
198
```

4. Number of people with ≥2 returned cheques and income < ₹50,000

```
print(" 🔹 People with ≥2 returned cheques and income < 50,000:")
loan_df.filter((col(" Returned Cheque") >= 2) & (col("income") < 50000)).count()
```

```
 🔹 People with ≥2 returned cheques and income < 50,000:
137
```

5. Number of people with ≥2 returned cheques and are single

```
print(" 🔹 People with ≥2 returned cheques and are single:")
loan_df.filter((col(" Returned Cheque") >= 2) & (col("marital_status") == "single")).count()
```

```
 🔹 People with ≥2 returned cheques and are single:
0
```

6. Number of people with monthly expenditure > ₹50,000

```
print(" 🔹 People with expenditure > 50,000 per month:")
loan_df.filter(col("Expenditure") > 50000).count()
```

```
 🔹 People with expenditure > 50,000 per month:
6
```

7. Number of members eligible for a credit card

```
from pyspark.sql.functions import col, when

# Add eligibility column based on rule: income ≥ 50000, age ≥ 21, returned cheques ≤ 1
loan_df = loan_df.withColumn(
    "eligible_credit_card",
    when(
        (col("income") >= 50000) &
        (col("Age") >= 21) &
        (col(" Returned Cheque") <= 1),
        "yes"
    ).otherwise("no")
)

# Count number of members eligible for credit card
print(" 🔹 Number of members eligible for credit card:")
loan_df.filter(col("eligible_credit_card") == "yes").count()
```

```
 🔹 Number of members eligible for credit card:
```

IN CREDIT.CSV DATA:

1. Credit card users in Spain

```
from pyspark.sql.functions import col

print("📌 Credit card users in Spain:")
credit_df.filter(col("Geography") == "Spain").show()
```

```
📌 Credit card users in Spain:
+---------+----------+---------+-----------+---------+------+---+------+---------+-------------+--------------+---------------+------+
|RowNumber|CustomerId|  Surname|CreditScore|Geography|Gender|Age|Tenure|  Balance|NumOfProducts|IsActiveMember|EstimatedSalary|Exited|
+---------+----------+---------+-----------+---------+------+---+------+---------+-------------+--------------+---------------+------+
|        2|  15647311|     Hill|        608|    Spain|Female| 41|     1| 83807.86|            1|             1|      112542.58|     0|
|        5|  15737888| Mitchell|        850|    Spain|Female| 43|     2|125510.82|            1|             1|       79084.1|     0|
|        6|  15574012|      Chu|        645|    Spain|  Male| 44|     8|113755.78|            2|             0|      149756.71|     1|
|       12|  15737173|  Andrews|        497|    Spain|  Male| 24|     3|      0.0|            2|             0|       76390.01|     0|
|       15|  15600882|    Scott|        635|    Spain|Female| 35|     7|      0.0|            2|             1|       65951.65|     0|
|       18|  15788218|Henderson|        549|    Spain|Female| 24|     9|      0.0|            2|             1|       14406.41|     0|
|       19|  15661507|  Muldrow|        587|    Spain|  Male| 45|     6|      0.0|            1|             0|      158684.81|     0|
|       22|  15597945| Dellucci|        636|    Spain|Female| 32|     8|      0.0|            2|             0|      138555.46|     0|
|       23|  15699309|Gerasimov|        510|    Spain|Female| 38|     4|      0.0|            1|             0|      118913.53|     1|
|       31|  15589475|  Azikiwe|        591|    Spain|Female| 39|     3|      0.0|            3|             0|      140469.38|     1|
|       34|  15659428|  Maggard|        520|    Spain|Female| 42|     6|      0.0|            2|             1|       34410.55|     0|
|       35|  15732963| Clements|        722|    Spain|Female| 29|     9|      0.0|            2|             1|      142033.07|     0|
|       37|  15788448|   Watson|        490|    Spain|  Male| 31|     3|145260.23|            1|             1|      114066.77|     0|
|       38|  15729599|  Lorenzo|        804|    Spain|  Male| 33|     7|  76548.6|            1|             1|       98453.45|     0|
|       41|  15619360|    Hsiao|        472|    Spain|  Male| 40|     4|      0.0|            1|             0|       70154.22|     0|
|       45|  15684171|  Bianchi|        660|    Spain|Female| 61|     5|155931.11|            1|             1|      158338.39|     0|
|       59|  15623944|    T'ien|        511|    Spain|Female| 66|     4|      0.0|            1|             0|        1643.11|     1|
|       63|  15702014|  Jeffrey|        555|    Spain|  Male| 33|     1| 56084.69|            2|             0|      178798.13|     0|
|       64|  15751208|  Pirozzi|        684|    Spain|  Male| 56|     8| 78707.16|            1|             1|       99398.36|     0|
|       73|  15812518|  Palermo|        657|    Spain|Female| 37|     0|163607.18|            1|             1|       44203.55|     0|
+---------+----------+---------+-----------+---------+------+---+------+---------+-------------+--------------+---------------+------+
only showing top 20 rows
```

2. Number of members who are eligible and active in the bank

```
from pyspark.sql.functions import col

# Count members who are eligible (CreditScore ≥ 650) and active (IsActiveMember = 1)
eligible_active_count = credit_df.filter(
    (col("CreditScore") >= 650) & (col("IsActiveMember") == 1)
).count()

print(f"📌 Number of members who are eligible and active in the bank: {eligible_active_count}")
```

```
📌 Number of members who are eligible and active in the bank: 2672
```

IN TRANSACTIONS FILE

1. Maximum withdrawal amount in transactions

```
from pyspark.sql.functions import lower, col, max

txn_df.filter(lower(col("TRANSACTION DETAILS")).rlike(".*(withdrawal|atm|pos|sett).*")) \
      .select(max("WITHDRAWAL AMT").alias("Max_Withdrawal_Amount")) \
      .show()
```

```
+--------------------+
|Max_Withdrawal_Amount|
+--------------------+
|               8.0E7|
+--------------------+
```

2. Minimum Withdrawal Amount of an Account

```
from pyspark.sql.functions import min

txn_df.filter(lower(col("TRANSACTION DETAILS")).rlike(".*(withdrawal|atm|pos|sett).*")) \
      .select(min("WITHDRAWAL AMT").alias("Min_Withdrawal_Amount")) \
      .show()
```

```
+--------------------+
|Min_Withdrawal_Amount|
+--------------------+
|                0.01|
+--------------------+
```

3. Maximum Deposit Amount of an Account

```
txn_df.filter(lower(col("TRANSACTION DETAILS")).rlike(".*(deposit|credit|income).*")) \
      .select(max("DEPOSIT AMT").alias("Max_Deposit_Amount")) \
      .show()
```

```
+------------------+
|Max_Deposit_Amount|
+------------------+
|      1.00150685E8|
+------------------+
```

4. Minimum Deposit Amount of an Account

```
txn_df.filter(lower(col("TRANSACTION DETAILS")).rlike(".*(deposit|credit|income).*")) \
      .select(min("DEPOSIT AMT").alias("Min_Deposit_Amount")) \
      .show()
```

```
+------------------+
|Min_Deposit_Amount|
+------------------+
|               9.0|
+------------------+
```

5. Sum of Balance in Every Bank Account

```
txn_df.groupBy("account_no") \
      .agg({"BALANCE AMT": "sum"}) \
      .withColumnRenamed("sum(BALANCE AMT)", "Total_Balance") \
      .show()
```

```
+------------+--------------------+
|  account_no|       Total_Balance|
+------------+--------------------+
|409000438611'|-2.49486577068339...|
|     1196711'|-1.60476498101275E13|
|     1196428'| -8.1418498130721E13|
|409000493210'|-3.27584952132095...|
|409000611074'|       1.615533622E9|
|409000425051'|-3.77211841164998...|
|409000405747'|-2.43108047067000...|
|409000362497'| -5.2860004792808E13|
|409000493201'|1.0420831829499985E9|
|409000438620'|-7.12291867951358...|
+------------+--------------------+
```

6. Number of Transactions on Each Date

```
txn_df.groupBy("VALUE DATE") \
      .count() \
      .withColumnRenamed("count", "Transaction_Count") \
      .orderBy("VALUE DATE") \
      .show()
```

```
+----------+-----------------+
|VALUE DATE|Transaction_Count|
+----------+-----------------+
|  1-Apr-17|                1|
|  1-Aug-15|               75|
|  1-Aug-16|               85|
|  1-Aug-17|               65|
|  1-Aug-18|              144|
|  1-Dec-15|               96|
|  1-Dec-16|              106|
|  1-Dec-17|               45|
|  1-Dec-18|               97|
|  1-Feb-16|               97|
|  1-Feb-17|               81|
|  1-Feb-18|               87|
|  1-Feb-19|               79|
|  1-Jan-15|                3|
|  1-Jan-16|               59|
|  1-Jan-18|               53|
|  1-Jan-19|               57|
|  1-Jul-15|               25|
|  1-Jul-16|              111|
|  1-Jul-17|              243|
+----------+-----------------+
only showing top 20 rows
```

7. List of Customers with Withdrawal Amount More Than ₹1,00,000

```
txn_df.filter((col("WITHDRAWAL AMT") > 100000)) \
      .select("account_no", "WITHDRAWAL AMT") \
      .distinct() \
      .orderBy("WITHDRAWAL AMT", ascending=False) \
      .show()
```

```
+-------------+--------------+
|   account_no|WITHDRAWAL AMT|
+-------------+--------------+
|     1196711'| 4.594475464E8|
|     1196711'| 4.482072231E8|
|409000438620'|         4.0E8|
|409000425051'|        3.54E8|
|     1196711'| 2.671403184E8|
|409000438611'|         2.4E8|
|     1196711'|       2.021E8|
|     1196711'|         2.0E8|
|409000438620'|         2.0E8|
|409000405747'|         1.7E8|
|     1196711'|        1.54E8|
|     1196711'|         1.5E8|
|     1196428'|         1.5E8|
|409000362497'| 1.413662392E8|
|409000362497'| 1.317762365E8|
|409000362497'| 1.316962119E8|
|409000362497'| 1.307105185E8|
|409000438611'|         1.3E8|
|409000362497'| 1.255964844E8|
|409000362497'| 1.221570889E8|
+-------------+--------------+
only showing top 20 rows
```